# Advanced Retrieval and RAG Architectures

pig7selene

September 5, 2026

**Abstract**

A one-pass Retrieval-Augmented Generation pipeline asks one query of one index and gives the resulting context to a generator. That interface is insufficient when a query needs reformulation, several evidence paths, or a representation of document structure. This chapter develops query transformation, multi-query and decomposed retrieval, HyDE, feedback-driven and multi-hop retrieval, hierarchy, graph-based retrieval, and adaptive routing. Its central concern is not adding stages indiscriminately, but selecting a richer retrieval policy only when a measurable limitation of the simple pipeline justifies its cost and failure surface.

## 1 Introduction

Chapter 26 established the elementary Retrieval-Augmented Generation (RAG) path:

$$\text{query} \longrightarrow \text{retrieve} \longrightarrow \text{context} \longrightarrow \text{generate}$$

Chapters 27–31 refined its components with semantic and sparse retrieval, vector indexing, segmentation, hybrid candidate generation, and reranking. The elementary path remains a strong baseline. It is low latency, has a small number of interfaces, and can be inspected as one query, one candidate set, and one context. It fails, however, when the user formulation is a poor retrieval key, when evidence is dispersed across several sources, or when the appropriate next search depends on an earlier result.

An **advanced retrieval architecture** changes one or more of these interfaces. It may transform the query before search, issue several complementary queries, separate a compound question into subproblems, retrieve again after inspecting evidence, move between levels of a document hierarchy, or traverse links in a graph. The common aim is to make the needed evidence reachable without asking the generator to invent a connection that retrieval never exposed.

More stages are not automatically better. Each transformation can drift from the user's information need; each additional search can add latency, duplicate evidence, and another opportunity for a model error to become a retrieval error. This chapter therefore treats advanced retrieval as a controlled policy over a retrieval budget. It does not develop Agent architectures, Tool Calling, general planning, LangChain, LangGraph, or a full RAG evaluation methodology. An LLM may participate in rewriting or routing, but the object of study remains the retrieval architecture.

## 2 Query Transformation and Parallel Retrieval

The question a user asks is not necessarily the string that should be passed to a retriever. Let $q_{\text{user}}$ denote the user-facing question and let $q_{\text{retr}}$ denote a retrieval-oriented query. **Query Rewriting** constructs

$$q_{\text{retr}} = \rho(q_{\text{user}}), \tag{1}$$

where $\rho$ may resolve a reference from dialogue history, remove conversational framing, make an entity explicit, or use terminology that is more likely to occur in the corpus. The rewrite is an interface transformation, not a new user request. Its contract must preserve the original information need: rewriting What did the report say about it?'' after a discussion of a named report may be necessary; silently replacing a question about a specific model version with a broader question about the product is not.

**Query Expansion** augments a retrieval query with alternative lexical or semantic cues. Synonyms, abbreviations, entity aliases, and domain terms can improve recall when the corpus and the user use

different vocabulary. Classical relevance feedback and query expansion distinguish global reformulations from local changes informed by initially retrieved documents [1]. Expansion has a symmetric risk: every added term or concept can make a result set broader in the wrong direction. This **query drift** is especially damaging in a hybrid system because an expanded lexical path can introduce a large candidate pool that appears plausible but no longer satisfies the original constraint.

**Multi-Query Retrieval** issues several formulations for one information need. Given

$$q_{\text{user}} \to (q_1, \ldots, q_m), \tag{2}$$

each $q_i$ retrieves a candidate set $C_{K(q_i)}$. The initial pooled set is

$$C_{\text{multi}(q_{\text{user}})} = \cup_{i=1}^{m} C_{K(q_i)}. \tag{3}$$

The union in Equation 3 must be deduplicated by retrieval-unit and source identity before it is passed to the reranking and context-selection stages of Chapter 31. The rank fusion methods from Chapter 30, including Reciprocal Rank Fusion, can combine the per-query result lists without assuming that their raw scores are comparable. Multiple formulations are useful when the corpus may use several names for the same concept. They cost $m$ retrieval operations, enlarge the candidate pool, and can yield several copies of the same evidence.

**Query Decomposition** solves a different problem. A complex information need is divided into distinct subproblems:

$$\begin{gathered}\text{complex question} \longrightarrow \text{subquery 1, subquery 2, ..., subquery } n \\ \longrightarrow \text{retrieve evidence for each subproblem} \longrightarrow \text{aggregate evidence}\end{gathered}$$

Multi-query retrieval expresses one intended search in several ways; decomposition identifies several searches whose answers must be combined. A question asking for the regulator of an organization and the date of that regulator's latest ruling may require two different evidence paths. Decomposition can make coverage explicit, but an incorrect split can omit a condition that links the subproblems or create unsupported intermediate assumptions. The answer generator must not be asked to treat independent subquery answers as mutually consistent without retaining their sources and scope.

| Strategy | What it changes | Principal cost or risk |
|---|---|---|
| Query rewriting or expansion | The query representation before a first retrieval pass | Intent loss or query drift |
| Multi-query retrieval | Several formulations of one information need | Extra searches, duplicate candidates, and fusion cost |
| Query decomposition | One compound need into distinct evidence tasks | Incorrect subproblems or lost dependencies |
| Iterative retrieval | Later queries conditioned on earlier evidence | Error propagation, loops, and growing latency |
| Hierarchical or graph retrieval | The search space and relations available to retrieval | Index-construction cost and structural errors |

Table 1: Advanced retrieval methods alter different interfaces in the basic RAG path. Their costs are not interchangeable: a query rewrite changes intent risk, whereas a graph index changes ingestion and traversal risk.

# 3 Hypothetical Documents and Retrieval Feedback

Hypothetical Document Embeddings (HyDE) change the representation used for dense retrieval rather than simply adding terms to the original query. A language model first writes a hypothetical passage $\tilde{d}$ that might answer $q_{\text{user}}$. A document encoder then embeds the hypothetical text and retrieves real corpus passages near that vector:

$$q_{\text{user}} \to \text{generate } \tilde{d} \to f(\tilde{d}) = e_{\text{HyDE}} \to \text{TopK}_{d \in \mathcal{D}} \, \text{sim}\big(e_{\text{HyDE}}, e_d\big). \tag{4}$$

The intuition is geometric. A short question and an answer-bearing passage can have different linguistic forms, while a generated passage can resemble the style and content pattern of corpus

passages more closely. HyDE uses the generated text as a retrieval representation, not as evidence. It need not be factually correct. The dense retrieval step is meant to ground the query in actual corpus items, and the hypothetical passage must never be presented as a source or merged into an answer as though it had been retrieved [2].

This separation also makes HyDE's limitations clear. Hallucinated entities or relations can steer $e_{\mathrm{HyDE}}$ toward the wrong neighborhood. The usefulness of the technique depends on the embedding model's geometry, the prompting policy, and the target corpus. It adds a generation step before retrieval and should be compared with a query-only baseline under the same candidate depth and latency budget. A plausible hypothetical document is not proof of a useful retrieval representation.

**Retrieval with feedback** closes a related loop. An initial result set can reveal an alias, a missing constraint, or an entity needed for a more precise follow-up query. In classical relevance feedback, judged or presumed-relevant results inform a revised query; modern systems may use a model to inspect retrieved metadata or passages and propose the revision. The important distinction is between evidence and control. A passage used to form the next query is a tentative signal, not automatically verified support for the eventual answer. The system should record which result informed the revision and should preserve the original query alongside every later one.

# 4 Iterative and Multi-Hop Retrieval

Some questions cannot be answered from a first result list because the next search key is itself contained in the retrieved evidence. Let $q_0 = q_{\mathrm{user}}$, let $E_t$ be the evidence retained after round $t$, and let $u$ be a query-update procedure. An iterative retrieval loop can be written as

$$q_{t+1} = u(q_0, E_1, \ldots, E_t), \quad E_{t+1} = \mathrm{Retrieve}(q_{t+1}). \tag{5}$$

Unlike Multi-Query Retrieval, the next query in Equation 5 is not available independently at the start. It can depend on an entity, relation, or contradiction exposed by a prior round. Work on interleaving retrieval and intermediate reasoning makes this dependency explicit for knowledge-intensive multi-step questions: what is retrieved next can depend on what earlier evidence has established [3].

**Multi-Hop Retrieval** is a particular iterative setting in which answering requires connected evidence. A first hop may retrieve an entity description; a relation mentioned there determines the next hop; the answer is supported only after the resulting pieces are combined. Multi-hop question-answering benchmarks such as HotpotQA make the need for multiple supporting documents and attribution visible [4]. Multi-hop dense retrieval formalizes a recursive retriever that conditions later retrieval on the question and previously retrieved passages [5].

The additional expressiveness creates an error-propagation path. If a first result names the wrong entity, a perfectly executed second search can be irrelevant to the original question. Branching over several first-hop candidates can increase recall but causes the candidate space to grow quickly. An iterative system therefore needs to retain provenance by round, limit branching, rerank candidate paths as well as individual passages where appropriate, and distinguish an uncertain intermediate hypothesis from an established source fact.

The retrieval loop should also have an explicit stopping rule. Let $\Delta_t$ denote an estimated marginal evidence gain of round $t$, $R_{\max}$ a maximum number of rounds, and $b_t$ the remaining retrieval budget. A conceptual policy is

$$\mathrm{stop}_t = 1 \quad \text{if } t = R_{\max} \ \text{ or } \ \Delta_t < \delta \ \text{ or } \ b_t \le 0. \tag{6}$$

The gain estimate in Equation 6 is only a controller signal; it is not a reliable statement that the answer is true. It might use new-source coverage, lack of score improvement, result redundancy, a learned confidence estimate, or a fixed budget. Stopping too early leaves an evidence chain incomplete; stopping too late produces an expensive loop of near-duplicate searches. The policy must make this latency–quality trade-off observable.

# 5 Hierarchical and Recursive Retrieval

Chapter 29 introduced Parent–Child Retrieval: index small child chunks for precise matching, then map a selected child to a larger parent section or local neighborhood before generation. This is already an advanced retrieval decision because the object that maximizes first-stage relevance is not necessarily the object that provides complete evidence. The parent expansion must preserve source identity, ordered spans, and the context budget; otherwise, a precise hit can become an opaque block of unrelated text.

**Hierarchical Retrieval** generalizes the idea to several levels:

$$\text{document} \longrightarrow \text{sections} \longrightarrow \text{subsections} \longrightarrow \text{chunks}$$
$$\text{retrieve or route at one or more levels} \longrightarrow \text{expand selected evidence}$$

For a book, paper, manual, or long report, a section-level representation can help locate the broad topic, while a child chunk can supply the exact statement. Hierarchy can also guide context expansion: retrieve a detailed passage, then attach its heading, parent scope, or a bounded neighboring span. This does not require a learned tree. A deterministic document hierarchy is valuable when it is extracted reliably and versioned with the source.

RAPTOR is a representative recursive design. It clusters and summarizes lower-level text to construct higher-level representations, permitting retrieval at several abstraction levels rather than only from contiguous leaf chunks [6]. The architecture can help a broad query reach a document-level theme and a narrow query reach a local passage. Its summaries, however, are derived artifacts: they can omit qualifiers, flatten disagreement, or become stale after an underlying source changes. A system should preserve links from every summary to the source spans it abstracts and should not treat a summary as an independent authoritative source.

# 6 Graph-Based Retrieval and Graph RAG

Flat vector retrieval treats each indexed chunk as an independent candidate except for similarity and any metadata filters. **Graph-Based Retrieval** adds an explicit relation structure. Let

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}) \tag{7}$$

be a retrieval graph, where nodes in $\mathcal{V}$ can represent entities, passages, documents, sections, or concepts, and edges in $\mathcal{E}$ can represent citations, hyperlinks, document containment, entity relations, or declared semantic links. A query can first retrieve seed nodes, then traverse selected edges to gather related evidence. The graph's usefulness depends on whether its edges represent a relation that is relevant to the task; connecting all semantically similar chunks indiscriminately merely turns a neighborhood into another source of redundancy.

A **Knowledge Graph** is a more structured instance in which nodes and edges have declared entity and relation semantics. A conceptual path is

$$\text{query} \longrightarrow \text{entity and relation identification} \longrightarrow \text{graph retrieval}$$
$$\longrightarrow \text{relevant nodes and edges} \longrightarrow \text{attributable textual context} \longrightarrow \text{generation}$$

Knowledge Graph retrieval can be useful for entity-centric and relational questions, where traversal makes a relation explicit that a dense similarity score may not expose. It is not a substitute for text retrieval. Entity resolution errors, incomplete extraction, relation-schema choices, and source updates can make a graph less complete or less current than the documents from which it was built. Graph evidence must retain its supporting source text and timestamps rather than reducing a contested relation to an untraceable edge.

**Graph RAG** names a family of RAG systems that combine generation with graph-structured text indexes or knowledge graphs; it is not one canonical algorithm. Some designs combine chunk retrieval with entity graphs and local traversal. Others construct community-level summaries for global questions over a corpus. The Graph RAG approach of Edge et al. builds an entity graph and community summaries, then uses those summaries to support query-focused synthesis over a large text collection [7]. That example illustrates a broader point: graph structure can change the unit retrieved and the

kind of question a system can support, but it also moves substantial complexity to ingestion, graph maintenance, and provenance management.

# 7 Adaptive Routing and Evidence Aggregation

Not every query deserves the same retrieval policy. An **adaptive retriever** or **router** can choose among no retrieval, one-pass retrieval, Multi-Query Retrieval, iterative retrieval, graph traversal, or a deeper candidate search. The router may use query length, explicit entities, ambiguous references, predicted complexity, source availability, initial candidate confidence, or a latency service-level objective. These are imperfect proxies. A short question can require several sources; a long question can be answerable from one precise passage. Routing should be evaluated against the selected policy's end-to-end evidence quality and cost, not merely against a complexity label.

Retrieval depth has several meanings and should be named precisely. It can mean the first-stage candidate depth $K$ from Chapter 31, the number of expansion levels in a hierarchy, the hop count in a graph, or the number of iterative rounds. Increasing any of these can expose more evidence, but it also raises retrieval, reranking, and context-construction costs. A system that silently changes depth in response to a model confidence score is difficult to reproduce; the route, thresholds, budgets, and observed stopping reason should be recorded with the answer trace.

Advanced retrieval commonly returns evidence from several paths. Before generation, the system must deduplicate spans, group passages by source and revision, retain a useful order, and assess whether the assembled context covers each required subproblem. More passages do not imply more support. Ten near-duplicate chunks provide less independent evidence than two complementary primary sources. Chapter 31′s distinction between reranking and final context selection remains essential: a retrieved or highly ranked item still may not fit the final context budget.

Conflicting evidence requires an explicit policy rather than a silent average. The context should preserve source identity, timestamp, version, authority signal, and the exact claim each passage supports. A system may prefer a current official source for a current factual question, surface the disagreement, request clarification, or decline to synthesize an unsupported conclusion. It should not concatenate incompatible claims merely because they are both relevant to a broad query. Conflict resolution is an evidence and provenance problem before it is a generation problem.

# 8 Failure Modes

The first class of failures comes from changing the query. A rewrite can remove a crucial qualifier; expansion can drift toward an associated but irrelevant topic; Multi-Query Retrieval can return redundant variants while hiding the original result in a large pool. Decomposition can assign a false premise to a subproblem. HyDE can encode hallucinated concepts that pull retrieval toward a wrong semantic neighborhood. These are not generic generator hallucinations: they are retrieval-control errors that must be inspected before the answer stage.

The second class comes from repeated or structured search. Iterative retrieval can amplify a wrong early entity, branch into an unmanageable candidate tree, or loop without gathering new evidence. Parent expansion can dilute a precise child hit with irrelevant parent text. Hierarchical summaries can lose a condition present in their leaves. Graph traversal can move from a correct seed through an irrelevant edge, while bad entity linking can attach the query to the wrong graph region. Stale graphs and summaries can conflict with a current source corpus.

Finally, complexity itself is a failure mode. A strong one-pass hybrid retriever with a suitable reranker can outperform a weakly controlled multi-stage pipeline. Every added component requires data, versioning, latency budget, evaluation slices, and a recovery path. Advanced retrieval should be introduced only when a measured failure of the simpler pipeline identifies the missing capability: vocabulary mismatch, insufficient candidate recall, missing multi-hop evidence, long-document scope, relational structure, or an unsuitable fixed-depth policy.

# 9 Implementation Contracts

The **query-control contract** must retain $q_{\text{user}}$, every rewritten, expanded, or decomposed query, the controller or prompt revision, declared intent-preservation policy, and the reason that each query was issued. It must bind multi-query fusion to the candidate identifiers and deduplication rules from Chapters 30 and 31. A HyDE implementation additionally needs the hypothetical-document prompt, generator revision, embedding model, and a guarantee that hypothetical text is never presented as retrieved evidence.

The **iterative contract** must define state retained across rounds, allowed query-update inputs, branch factor, maximum rounds, candidate and reranker depths per round, stopping policy, timeout, and fallback behavior. It should record evidence by round and distinguish retrieved documents from inferred intermediate claims. Tests should include an intentionally wrong first-hop entity, redundant feedback, a query requiring two independent sources, and a request that must stop because its budget is exhausted.

The **structure contract** must version parent–child links, hierarchy extraction, summary derivations, graph schema, entity-resolution model, edge types, traversal policy, and source-to-node provenance. It must specify how source revisions delete or rebuild nodes, edges, summaries, and indexes. Permission restrictions propagate through every expansion and traversal: an eligible child must not permit the system to expose an ineligible parent or neighboring node.

The **evaluation contract** should compare every advanced route with a fixed one-pass baseline under the same corpus revision. It should report candidate recall, evidence coverage by subproblem, duplicate rate, source diversity, conflict handling, answer support, retrieval and reranking latency, generation-context size, and total cost. Slice tests should include ambiguous language, aliases, compound questions, multi-hop chains, long documents, incorrect entity links, stale graph relations, conflicting source versions, and cases where additional retrieval produces no new useful evidence.

# 10 Summary

Advanced retrieval improves on a single query and single search pass by changing the query, repeating retrieval in response to evidence, or exposing document and graph structure. Query rewriting, expansion, and Multi-Query Retrieval address vocabulary and formulation mismatch; decomposition, iterative retrieval, and multi-hop retrieval address evidence dependencies; Parent–Child, hierarchical, recursive, and graph-based methods change how the corpus is represented and traversed.

These strategies are controlled retrieval policies, not interchangeable features. A useful system preserves the original query, source provenance, and evidence history; routes only when a simple baseline cannot meet the need; and stops when additional retrieval no longer offers enough expected evidence to justify its cost. The next RAG chapter can evaluate such architectures, but meaningful evaluation begins with the observable interfaces and failure boundaries established here.

# References


[1] C. D. Manning, P. Raghavan, and H. Schutze, *Introduction to Information Retrieval*. Cambridge University Press, 2008. doi: 10.1017/CBO9780511809071.

[2] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 1762–1777. doi: 10.18653/v1/2023.acl-long.99.

[3] H. Trivedi, N. Balasubramanian, T. Khot, and A. Sabharwal, "Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2023, pp. 10014–10037. doi: 10.18653/v1/2023.acl-long.557.

[4] Z. Yang *et al.*, "HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2018, pp. 2369–2380. doi: 10.18653/v1/D18-1259.

[5] W. Xiong *et al.*, "Answering Complex Open-Domain Questions with Multi-Hop Dense Retrieval," in *International Conference on Learning Representations*, OpenReview.net, 2021. [Online]. Available: https://openreview.net/forum?id=EMHoBG0avc1

[6] P. Sarthi, S. Abdullah, A. Tuli, S. Khanna, A. Goldie, and C. D. Manning, "RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval," *arXiv preprint arXiv:2401.18059*, 2024, doi: 10.48550/arXiv.2401.18059.

[7] D. Edge *et al.*, "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024, doi: 10.48550/arXiv.2404.16130.